

GROUP 6

STEPHEN MWANGI

ENE221-0100/2022

SIDNEY NGIRU

ENE221-0138/2022

EUNICE KIMANI

ENE221-0101/2022

OSCAR MWANGI

ENE221-0093/2022

MICHAEL NJUGUNA

ENE221-0096/2022

2. Milestone 3 – Distributed Architecture

The system must be transformed from a simple prototype into a **proper distributed architecture**.

The possible architectural styles include:

- **Client-server** – one central server, many clients.
- **Multi-tier** – layered structure (e.g. Edge → Core → Cloud).
- **Service-oriented** – system built from independent services.
- **Microservice-based** – fine-grained, independently deployable services.

The system should evolve toward:

Edge ↔ Core ↔ Cloud

Deliverables required:

- Architectural design
- Deployment architecture
- Service decomposition
- Scalability analysis
- Edge vs Core trade-off analysis

How the requirements are met

Requirement

How it is tackled

Evaluate four architectural styles

Client-server = too flat; Multi-tier = partially true; Service-oriented /

Requirement	How it is tackled
	microservice = chosen. It combines a multi-tier deployment topology with microservice decomposition inside each tier .
Architectural design	Formalised as a layered, service-oriented system. Each node type hosts distinct independently addressable services.
Deployment architecture	Diagram shows: Users/Devices → edge-1/edge-2 → ran-1 → core-1 → cloud-1 Topology unchanged from M1 but now named precisely.
Service decomposition	Every node advertises services over the wire (<code>service_info</code> message). Table: EDGE = SessionHandling, LocalQoSScheduling; RAN = RadioResourceControl, LocalQoSScheduling; CORE = ResourceOrchestration, MobilityManagement, Coordination; CLOUD = Analytics, LongTermStorage, PolicyManagement.
Scalability analysis	Same 90-task workload, concurrency 6, varying Edge nodes (1, 2, 4). Results: rejection drops from 93.3% → 32.2% → 7.8% . Latency rises slightly (24.3 → 24.4 → 31.4 ms). Throughput is non-monotonic (171.4 → 236.8 → 175.7 tasks/s) because concurrency was fixed.
Edge vs Core trade-off	Edge-only: 237.0 tasks/s, 24.6 ms, 33.3% loss. Core-only: 201.2 tasks/s, 26.9 ms, 11.1% loss. Finding: two weak edge nodes out-throughput one stronger core, but core is more reliable per task. Motivates QoS-based placement (URLLC → edge, eMBB → core).

3. Milestone 4 – Distributed Algorithms and Coordination

What the project brief requires

The system must introduce **distributed coordination**.

You must implement an appropriate mechanism such as:

- **Leader election** – choosing one node to coordinate.
- **Consensus** – all nodes agree on a value.
- **Distributed synchronization** – keeping nodes in step.
- **Logical clocks** – ordering events without a shared physical clock.
- **Event ordering** – reconstructing a consistent order of events.

You must evaluate:

- Coordination overhead
- Message complexity
- Synchronization delay
- Consistency
- Scalability

Deliverables required:

- Distributed algorithm implementation
- Algorithm analysis
- Complexity analysis
- Experimental results

How the requirements are met

Requirement	How it is tackled
Leader election	Implemented Bully algorithm in <code>common/coordination.py::BullyElection</code> . Uses the existing TCP/JSON transport. Ranking (<code>election_id</code>) mirrors resource capacity, so Cloud is preferred coordinator.
Experimental results	Three elections from different starting points: edge-1 (lowest rank), ran-1 (mid rank), cloud-1 (highest rank). All elected cloud-1 . All 5 nodes consistently agreed.
Message complexity	Message count scales with how many nodes outrank the initiator. Worst

Requirement	How it is tackled
	case $O(n^2)$, best case $O(n)$. Edge-1 took most rounds/messages; cloud-1 won immediately with one broadcast round.
Consistency	After every run, all 5 nodes were independently queried (leader_query) and unanimously agreed on the same leader. A real bug was found and fixed: the winning node never told itself it had won.
Synchronization delay	Sub-3 ms for all three scenarios because communication is local-network TCP.
Logical clocks	Every message updates a Lamport clock per node. Three concurrent clients sent interleaved pings. Sorting by Lamport timestamp reconstructs one consistent total order. Example: lamport=8 client-B→edge-1, lamport=9 client-C→edge-1, etc.
Event ordering	Demonstrated causal, consistent ordering of distributed events without a shared physical clock.
Coordination overhead	Measured in rounds, messages, and time for each election.
Scalability	Partially evaluated through different initiators, but only on the same 5-node topology.

4. Cross-cutting requirements

4.1 Failure-Driven Engineering Log (Week 3–4)

The project brief requires a log answering:

What changed? What failed? Why? How fixed? Alternative? Learned?

The report includes this for both the main system and the Big Data extension.

Main system failure:

- **What failed:** First leader-election run reported cloud-1 as inconsistent — every other node recorded cloud-1 as leader, but cloud-1 itself showed `current_leader: None`.
- **Why:** `BullyElection._announce_coordinator()` broadcast to every peer but not to itself.
- **Fix:** Send the same “coordinator” message to itself via loopback. Verified with consistency check across all 5 nodes.
- **Alternative considered:** Special-case local update without network round-trip. Rejected because it bypasses the normal message handler.
- **Learned:** “Broadcast to all peers” is a common off-by-one distributed bug. Consistency checks that poll every node are essential.

Big Data extension failures:

- Spark processed 100,000 rows as a single partition - fixed by explicit `.repartition(8)`.
- `pandas 3.x groupby().apply()` dropped `service_class` - fixed by explicit per-class loop and `pd.concat`.

4.2 Reproducibility

The project brief requires documenting hardware, OS, language, frameworks, versions, configuration, dataset, seeds, network parameters, and experimental parameters.

Report provides:

- Same environment as Assignment 1: Python 3, standard library only.
- Extra dependencies for Big Data: `pyspark`, `pandas`, `numpy`, `pyarrow`, plus a JDK.
- Topology, election ranks, and service assignments in `common/topology.py`.
- Seeds: M3 seed=11, M4 inherited, Big Data seed=2026, live sample seed=99.
- Run order: `milestone3_architecture` - `milestone4_coordination` - `assignment2_bigdata_extension`.

4.3 Big Data & Power BI Extension

This is an additional layer built on top of M3 and M4.

What it adds:

- **Apache Spark (PySpark, local mode)** for batch processing.
 - **Lambda Architecture:** Batch layer (Spark) + Speed layer (live node platform) + Serving layer (CSVs) + Presentation (Power BI).
 - **100,000-row dataset** processed in 8 partitions.
 - Batch results confirm **ran-1 bottleneck** at full scale: ran-1 receives fewer tasks than edge nodes but has roughly double the CPU demand because it absorbs 100% of eMBB traffic.
 - Speed layer: 600-task stratified sample → ~783 tasks/s, ~11.2 ms mean latency, ~46% rejection.
 - Power BI dashboard guide produces a 5-page dashboard: Scalability, Coordination, Architecture, Batch Layer (100K rows), Speed Layer (live sample).
-